

Running PyFlowCL

NOTE: For *Summit* instructions, see the [supercomputing](#) page.

PyFlowCL is structured as a Python library with submodules. The main function, `PyFlowCL.run()`, is located in `PyFlowCL.py`. Python-based **driver scripts** configure and invoke each instance of PyFlowCL. The basic tasks each driver script performs are:

1. Create an object containing input parameters and functions (`inputConfig`)
2. Start PyFlowCL via `PyFlowCL.run(inputConfig)`
3. Process PyFlowCL return data (if applicable)

Several example driver scripts are available in the verification cases. A good way to get started is to copy a driver script to your running directory. Because the example scripts are included in the PyFlowCL repo, they use relative paths to the PyFlowCL source tree. You may change this to an absolute path, e.g., if your running directory is on a scratch disk:

```
sys.path.append('/<path to PyFlow>/src')
from PyFlowCL import PyFlowCL, Grid
```

To execute a driver script in serial on the command line, use

```
python -u <driver_script.py>
```

To execute in parallel on **N** processors, use

```
mpirun -np N python -u <driver_script.py>
```

To enable OpenMP threading (CPU-only runs), specify the `OMP_NUM_THREADS` environment variable (i.e. `export OMP_NUM_THREADS=X` where "X" is the maximum number of threads OpenMP can use). GPU acceleration should occur automatically on GPU systems if PyTorch was compiled with CUDA support. To see the current value set, enter the command `echo $OMP_NUM_THREADS`.

In certain advanced cases, PyFlowCL calls back to the driver script during the simulation, e.g., to retrieve the name of target data files during adjoint-based model training. Since PyFlowCL is designed to be scripted by Python, driver scripts can get as fancy as the user wants.

NOTE: It is highly recommended that you do not run PyFlowCL in the source tree. Most simulations will save a lot of data and so are usually best performed on a scratch disk.